UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 4

Student Name: Aryan **UID:25MCI10082**
Branch: MCA (AI&ML) **Section/Group:1/A**
Semester:2nd **Date of Performance:04/02/2026**
Subject Name: DBMS LAB **Subject Code: 25CAP-652**

1. Aim of the Session

To understand and implement iterative control structures in PostgreSQL conceptually, including FOR loops, WHILE loops, and basic LOOP constructs, for repeated execution of database logic.

2. Software Requirements

- PostgreSQL (Database Server)
- pgAdmin
- Windows Operating System

3. Objective of the Session

- To understand why iteration is required in database programming
- To learn the purpose and behavior of FOR, WHILE, and LOOP constructs
- To understand how repeated data processing is handled in databases
- To relate loop concepts to real-world batch processing scenarios
- To strengthen conceptual knowledge of procedural SQL used in enterprise systems

4. Practical / Experiment Steps

Prerequisite Understanding

• Students should first understand that **iterative control structures** are executed inside **PL/pgSQL blocks**, not in normal SQL queries

• Students should create a table that stores **multiple records** so that loop execution over rows can be demonstrated

• The table should contain:

- A unique identifier
- A descriptive attribute (such as name or category)
- A numeric value to be processed repeatedly

5. Procedure of the Practical

(i) Start the system and log in to the computer.
(ii) Open PostgreSQL software.
iii) Create and select the database.

Create database Practical4;

(iv) Create table using DDL command.

```
CREATE TABLE employee (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    salary NUMERIC(10,2)
);
```

(v) Insert records into the table.

```
INSERT INTO employee (emp_name, salary) VALUES
```

('Amit Sharma', 45000.00),

('Neha Verma', 52000.50),

('Rahul Mehta', 60000.00),

('Priya Singh', 48000.75),

('Karan Gupta', 55000.00),

('Anjali Rao', 62000.25);

(vi) Display all records.

select*from employee;

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	1	Amit Sharma	45000.00
2	2	Neha Verma	52000.50
3	3	Rahul Mehta	60000.00
4	4	Priya Singh	48000.75
5	5	Karan Gupta	55000.00
6	6	Anjali Rao	62000.25

Step 1: FOR Loop – Simple Iteration

```
DO $$
BEGIN
    FOR i IN 1..6 LOOP
        RAISE NOTICE 'Iteration number: %', i;
    END LOOP;
END $$;
```

```
NOTICE:  Iteration number: 1
NOTICE:  Iteration number: 2
NOTICE:  Iteration number: 3
NOTICE:  Iteration number: 4
NOTICE:  Iteration number: 5
NOTICE:  Iteration number: 6
DO
```

Step 2: FOR Loop with Query (Row-by-Row Processing)

```
DO $$
DECLARE
   emp RECORD;
BEGIN
   FOR emp IN SELECT emp_id, emp_name FROM employee LOOP
      RAISE NOTICE 'Employee ID is % and Name is %', emp.emp_id,
   emp.emp_name; END LOOP;
END $$;
```

```
NOTICE:  Employee ID is 1 and Name is Amit Sharma
NOTICE:  Employee ID is 2 and Name is Neha Verma
NOTICE:  Employee ID is 3 and Name is Rahul Mehta
NOTICE:  Employee ID is 4 and Name is Priya Singh
NOTICE:  Employee ID is 5 and Name is Karan Gupta
NOTICE:  Employee ID is 6 and Name is Anjali Rao
DO

Query returned successfully in 80 msec.
```

Step 3: WHILE Loop – Conditional Iteration

```
DO $$
DECLARE
   counter INT := 1;
BEGIN
   WHILE counter <= 5 LOOP
      RAISE NOTICE 'Counter is %', counter;
      counter := counter + 1;
   END LOOP;
END $$;
```

```
NOTICE:  Counter is 1
NOTICE:  Counter is 2
NOTICE:  Counter is 3
NOTICE:  Counter is 4
NOTICE:  Counter is 5
NOTICE:  Counter is 6
DO

Query returned successfully in 58 msec.
```

Step 4: LOOP with EXIT WHEN

```
DO $$
DECLARE
   x INT := 1;
BEGIN
   LOOP
      RAISE NOTICE 'Value is %',
```

```
    x; x := x + 1;
    EXIT WHEN x > 6;
  END LOOP;
END $$;
```

```
NOTICE:  Value is 1
NOTICE:  Value is 2
NOTICE:  Value is 3
NOTICE:  Value is 4
NOTICE:  Value is 5
NOTICE:  Value is 6
DO

Query returned successfully in 77 msec.
```

Step 5: Salary Increment Using FOR Loop

```
DO $$
DECLARE
  emp RECORD;
BEGIN
  FOR emp IN SELECT emp_id, salary FROM employee LOOP
    UPDATE employee
    SET salary = salary * 1.25
    WHERE emp_id = emp.emp_id;
  END LOOP;
END $$;
select*from employee;
```

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	1	Amit Sharma	61875.00
2	2	Neha Verma	71500.69
3	3	Rahul Mehta	82500.00
4	4	Priya Singh	66001.04
5	5	Karan Gupta	75625.00
6	6	Anjali Rao	85250.35

Step 6: Combining LOOP with IF Condition

```
DO $$
DECLARE
    emp RECORD;
BEGIN
    FOR emp IN SELECT emp_name, salary FROM employee LOOP
        IF emp.salary > 70000 THEN
            RAISE NOTICE '% is a High Earner', emp.emp_name;
        ELSE
            RAISE NOTICE '% is a Regular Employee', emp.emp_name;
        END IF;
    END LOOP;
END $$;
```

```
NOTICE:  Amit Sharma is a Regular Employee
NOTICE:  Neha Verma is a High Earner
NOTICE:  Rahul Mehta is a High Earner
NOTICE:  Priya Singh is a Regular Employee
NOTICE:  Karan Gupta is a High Earner
NOTICE:  Anjali Rao is a High Earner
DO

Query returned successfully in 75 msec.
```

6. I/O Analysis (Input / Output)

Input:

- Employee or sample records inserted into a database table for iterative processing
- PL/pgSQL DO block containing procedural logic
- FOR loop constructs for fixed-range and query-based iteration
- WHILE loop with condition-based execution
- LOOP construct with explicit EXIT WHEN condition
- IF–ELSE conditions used inside loops for decision making
- UPDATE statements executed repeatedly within loop structures

Output:

- Repeated execution of SQL statements based on loop conditions
- Row-by-row processing of table records using FOR loops
- Conditional messages displayed during each iteration
- Successful salary updates or value modifications through iterative logic
- Proper termination of loops based on defined conditions
- Correct execution of procedural SQL demonstrating iteration control

Screenshots of execution and obtained results are attached.

7. Learning Outcomes

• Understood the need for iterative control structures in database programming

• Learned the usage of FOR, WHILE, and LOOP constructs in PostgreSQL

• Gained knowledge of executing repeated logic using PL/pgSQL

• Understood row-by-row processing and conditional execution in databases

• Developed foundational skills for writing procedural SQL in real-world applications